

STAT 413/513: Methods of Data Analysis III

Lab 7: May 13, 2025

Objectives

- Exploratory Plots for Binomial Count Data
- Performing Binomial Logistic Regression
- Performing Quasi-Binomial Logistic Regression

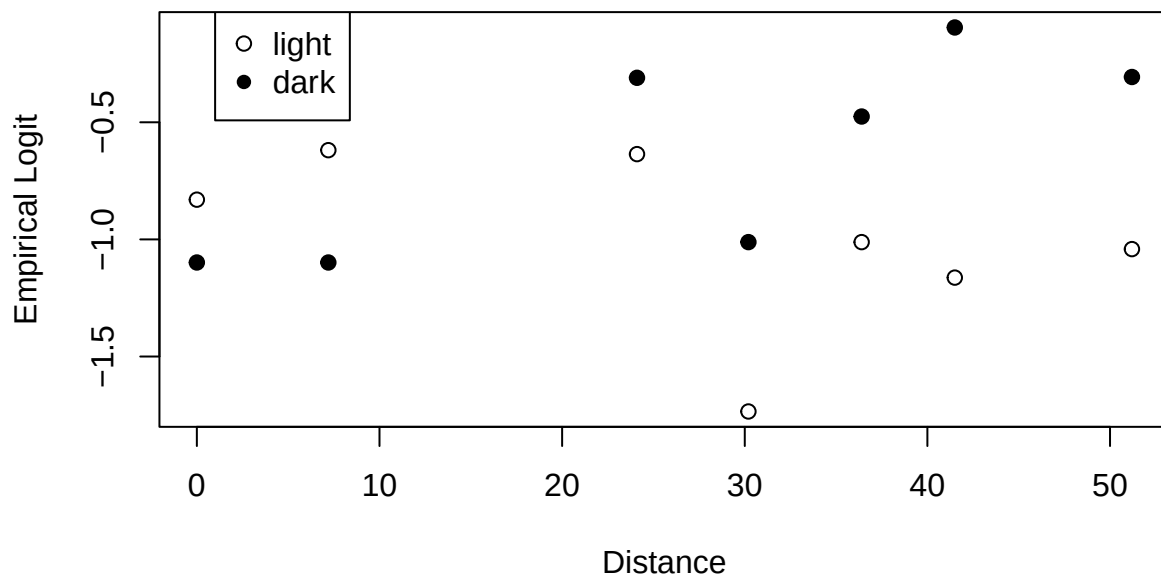
Exploratory Plots for Binomial Count Data

For this lab, we'll use the second case study in Chapter 21—the natural selection data:

```
library(Sleuth3)
attach(case2102)
```

We start by creating a plot of the empirical logits versus distance:

```
logits <- log(Removed/(Placed-Removed))
plot(Distance, logits, ylab="Empirical Logit")
points(Distance[Morph=="dark"], logits[Morph=="dark"], pch=16)
legend(x = 0, legend=c("light", "dark"), pch=c(1, 16))
```



It looks like the relationship between empirical logits and distance is different for the two morphs (suggesting an interaction). As such, we will include an interaction term in our model.

Performing Binomial Logistic Regression

The key difference between fitting a logistic regression model for **binomial** data and one for **binary** data in R is in specifying the response. We have to create a matrix with two columns—the first column contains the binomial counts (numbers of successes) and the second column contains the sample sizes minus the binomial counts (numbers of failures). For the natural selection data, we fit a binomial logistic regression model as follows:

```
resp <- cbind(Removed, (Placed-Removed))
dark <- ifelse(Morph=="dark",1,0)
binom.mod <- glm(resp~dark*Distance,family=binomial(link="logit"))
summary(binom.mod)

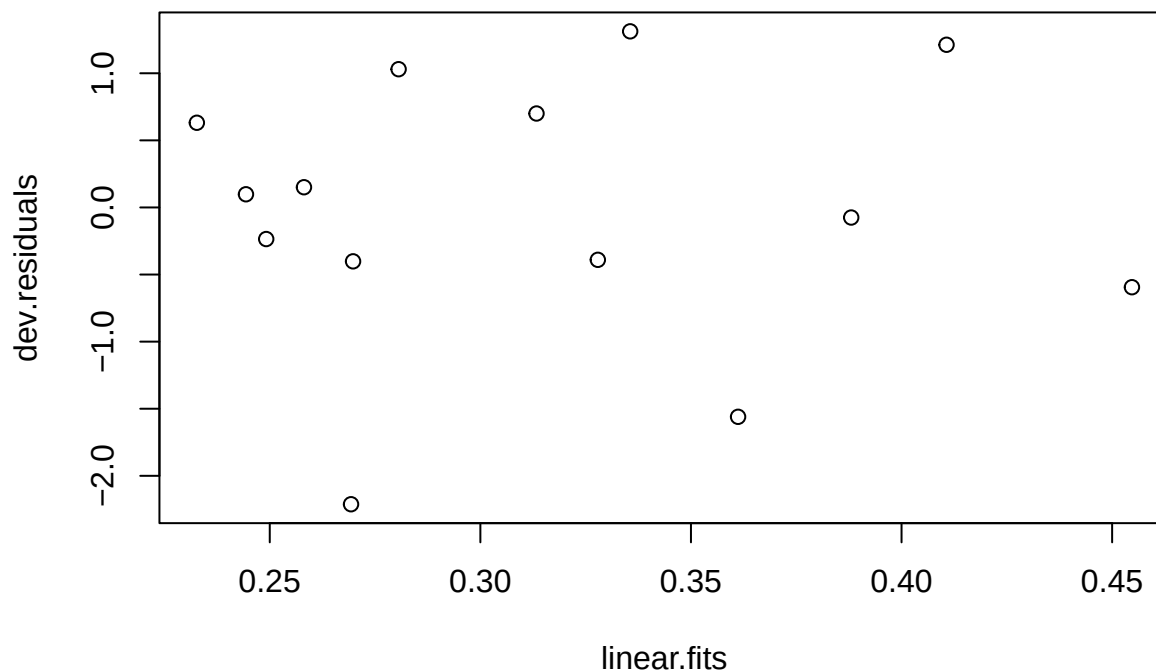
##
## Call:
## glm(formula = resp ~ dark * Distance, family = binomial(link = "logit"))
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.717729   0.190205  -3.773 0.000161 ***
## dark         -0.411257   0.274490  -1.498 0.134066
## Distance     -0.009287   0.005788  -1.604 0.108629
## dark:Distance  0.027789   0.008085   3.437 0.000588 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 35.385  on 13  degrees of freedom
## Residual deviance: 13.230  on 10  degrees of freedom
## AIC: 83.904
##
## Number of Fisher Scoring iterations: 4
```

Notice that in the summary output, the **dispersion parameter** is taken to be equal to one. This parameter can only be different from one if we fit a quasi-binomial model.

We can check the deviance residuals for evidence of overdispersion:

```
linear.fits <- fitted.values(binom.mod)
dev.residuals <- residuals(binom.mod, type = "deviance")
plot(x = linear.fits, y = dev.residuals, main = "Residuals vs Fitted Values")
```

Residuals vs Fitted Values



We can see a few potential outliers (particularly the observation with a deviance residual less than -2).

We can use the residual deviance in the model output to perform the deviance goodness-of-fit test:

```
pchisq(13.230, df = 10, lower.tail = F)
```

```
## [1] 0.2110951
```

Note that the degrees of freedom for the Chi-square test statistic are $df = n - p - 1 = 10$ and that they are provided in the model summary. With a p-value of 0.21, we do not have evidence that the binomial logistic regression model is a poor fit to the data. However, given our concerns about dependence between the removal of the moths on the same tree (as discussed in lecture), we still decide to account for overdispersion.

Performing Quasi-Binomial Logistic Regression

To fit a quasi-binomial model in R, we specify `family=quasibinomial(link="logit")` in the `glm` function:

```
quasibinom.mod <- glm(resp~dark*Distance,family=quasibinomial(link="logit"))
summary(quasibinom.mod)
```

```
##
## Call:
## glm(formula = resp ~ dark * Distance, family = quasibinomial(link = "logit"))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.717729   0.214423  -3.347   0.0074 **
## dark         -0.411257   0.309439  -1.329   0.2134
## Distance     -0.009287   0.006525  -1.423   0.1851
```

```
## dark:Distance 0.027789 0.009115 3.049 0.0123 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasibinomial family taken to be 1.270859)
##
## Null deviance: 35.385 on 13 degrees of freedom
## Residual deviance: 13.230 on 10 degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 4
```

We see that the estimated dispersion parameter in the quasibinomial model is 1.27. (Note that this estimate is different than what we get using the formula from class, namely $\hat{\psi} = deviance/(n-p-1) = 13.230/10 = 1.323$, though either estimate is acceptable.) Since this is greater than 1, it is indicative of potential **extra-binomial variation/overdispersion**.

Compare the summary output for binom.mod and quasibinom.mod. Notice that the coefficient estimates are the same, but that since the dispersion parameter in quasibinom.mod is greater than one, the standard error estimates for each of the coefficient estimates are all larger in quasibinom.mod than in binom.mod. Even though the conclusions are the same from the two models, they are not as strong under the quasi-binomial model.

Drop-in-Deviance F Test

Now fit one more model—removing the interaction term—and compare the models using a drop in deviance test. In the quasi-Binomial case, we perform an F -test to account for overdispersion. We can do this automatically in R with the `anova()` function by specifying `test="F"`:

```
quasibinom.mod.reduced <- glm(resp~dark+Distance,family=quasibinomial(link="logit"))
summary(quasibinom.mod.reduced)
```

```
##
## Call:
## glm(formula = resp ~ dark + Distance, family = quasibinomial(link = "logit"))
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.136742  0.235365  -4.830 0.000528 ***
## dark         0.404052  0.209269   1.931 0.079680 .
## Distance     0.005314  0.006008   0.884 0.395404
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasibinomial family taken to be 2.25436)
##
## Null deviance: 35.385 on 13 degrees of freedom
## Residual deviance: 25.161 on 11 degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 4
anova(quasibinom.mod.reduced, quasibinom.mod, test="F")
```

```
## Analysis of Deviance Table
##
```

```
## Model 1: resp ~ dark + Distance
## Model 2: resp ~ dark * Distance
##   Resid. Df Resid. Dev Df Deviance      F Pr(>F)
## 1      11      25.161
## 2      10      13.230  1   11.931  9.3885 0.01196 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Note that we passed two *quasi*-binomial models (as opposed to binomial models) to `anova()` to apply this test in R.) We see that the p -value is 0.01196, giving us evidence against the null hypothesis.

It is also worth checking these results against the results we get performing the adjustment by hand. Our estimate of the dispersion parameter is $\hat{\psi} = Deviance_{full}/df_{full} = 13.230/10 = 1.323$, so the test statistic is $F = \frac{(25.161 - 13.230)/1}{1.323} = 9.02$. The p -value is $P(F_{1,10} > 9.02) = 0.0132679$, which we compute using the `pf` function:

```
pf(9.02, df1 = 1, df2 = 10, lower.tail = F)
```

```
## [1] 0.01326791
```

Again, the results are slightly different because we are using a different estimate of the deviance than R does.

Since our p -value is less than 0.05, we reject the null hypothesis that the coefficient for the $\text{dark} \times \text{distance}$ interaction is 0. In other words, this test provides convincing evidence that the interaction term is needed in the model, which means that the relative odds of being predated for the two morphs change with distance from Liverpool.

t -test for a single coefficient

For the sake of practice, we can use a t -test with the quasi-binomial model to test whether the coefficient for the interaction, β_3 , is 0. One way is to look at the `quasibinom.mod` summary:

```
summary(quasibinom.mod)
```

```
##
## Call:
## glm(formula = resp ~ dark * Distance, family = quasibinomial(link = "logit"))
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.717729   0.214423  -3.347   0.0074 **
## dark         -0.411257   0.309439  -1.329   0.2134
## Distance     -0.009287   0.006525  -1.423   0.1851
## dark:Distance  0.027789   0.009115   3.049   0.0123 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasibinomial family taken to be 1.270859)
##
##      Null deviance: 35.385  on 13  degrees of freedom
## Residual deviance: 13.230  on 10  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 4
```

which gives us a p -value of 0.0123.

Another way is to adjust the test statistic by-hand with our estimate, $\hat{\psi} = 1.323$, of the dispersion parameter.

Our estimate of the standard error is

$$SE(\hat{\beta}_3^{QB}) = \sqrt{\hat{\psi}} SE(\hat{\beta}_3^{MLE}) = \sqrt{1.323} \times 0.008085 = 0.0092995$$

(Note: the standard error for the MLE, $SE(\hat{\beta}_3^{MLE}) = 0.008085$, is from `summary(binom.mod)`.) Therefore the test statistic is

$$t = \frac{0.0278}{0.0093} = 2.99$$

The sampling distribution is t_{10} , so the two-sided p -value is

$$p\text{-value} = 2 \times Pr(t_{10} > |2.99|) = 0.014$$

We can compute this p -value in R with the `pt()` function:

```
2*pt(2.99, df = 10, lower.tail = FALSE)
```

```
## [1] 0.01357366
```

As with the F -test, the p -value from our t -test is less than 0.05, so we (again) reject the null hypothesis that $\beta_3 = 0$.

Constructing a CI

We can also use the quasi-binomial model to construct a 95% confidence interval for β_3 . One way is to use the quasi-binomial model estimate of the standard error from R (from `summary(quasibinom.mod)`), which gives us the CI

$$\hat{\beta}_3 \pm t_{n-p-1}(\alpha/2) SE(\hat{\beta}_3^{QB}) = 0.0278 \pm 2.228 \times 0.0091 = (0.0075252, 0.0480748)$$

Note that the degrees of freedom are $df = n - p - 1 = 10$, $\alpha = 0.05$, and we can obtain the upper-0.025 quantile of a t_{10} distribution with `qt()`:

```
qt(0.025, df = 10, lower.tail = FALSE)
```

```
## [1] 2.228139
```

Here's some R code to compute this CI:

```
0.0277 + c(-1, 1)*2.228*0.0093
```

```
## [1] 0.0069796 0.0484204
```

Alternatively, we can make the correction for overdispersion by hand. Recall that our adjusted estimate of the standard error is 0.0093. Therefore our 95% CI is:

$$\hat{\beta}_3 \pm t_{n-p-1}(\alpha/2) SE(\hat{\beta}_3^{QB}) = 0.0277 \pm 2.228 \times 0.0093 = (0.0069796, 0.0484204)$$